

hardo. ko kisi ek symbol se replace

Posting list compression :

term :	12	13	15	120	127	510	512	514
gap list :	12	1	2	115	7	383	2	2

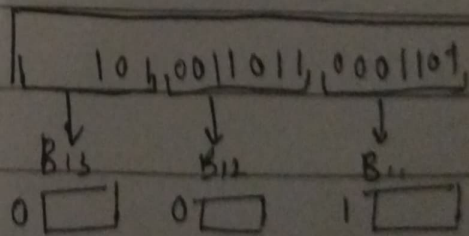
If term is very frequent then space between posting list is less and vice versa.

→ Encoding gap list is easier than posting values.
because small value : small space.

To encode gap list :

1) Variable byte encoding :

doc Id → 4 bytes.



Ex 52 0000000000000101011
↓
\$

2) Variable bit encoding: (Y codes).

Y code of a number: unary length of offset + offset of number.

offset: truncate 1st bit.

example: $\rightarrow 2$

10 0 10 100

$\rightarrow 5$

101 01 110 110,01

~~101~~

~~110 01~~

$\rightarrow 64$

1000000 000000 111110 11111000000

~~1000000~~

~~111110 000000~~

2 3 2 5 $\rightarrow$ gap list.
↓ ↓ ↓ ↓
100 11001 100 11001

variable list

VBEncodeNumbers(n)

- 1) bytes $\leftarrow \{\}$
- 2) while true
- 3) do Prepend (bytes, $n \bmod 128$)
- 4) if $n < 128$.
- 5) then break
- 6) $n \leftarrow n \div 128$
- 7) bytes [Length (bytes)] += 128.
- 8) return bytes.

VBEncode (numbers)

- 1) bytestream $\leftarrow \{\}$
- 2) for each $n \in \text{numbers}$
- 3) do bytes $\leftarrow \text{VBEncodeNumbers}(n)$
- 4) bytestream $\leftarrow \text{Extend}(\text{bytestream}, \text{bytes})$
- 5) return bytestream

VBDecode (bytestream)

- 1) numbers $\leftarrow \{\}$
- 2) $n \leftarrow 0$
- 3) for $i \leftarrow 1$ to LENGTH (bytestream)
- 4) do if bytestream[i] < 128
- 5) then $n \leftarrow 128 \times n + \text{bytestream}[i]$
- 6) else $n \leftarrow 128 \times n + (\text{bytestream}[i] - 128)$
- 7) Append (numbers, n).
- 8) $n \leftarrow 0$
- 9) return numbers.

gamma Code (γ code).

24 → 11000 → Binary

offset length in binary code + offset.

1000

11110 1000

111000 111010111110110 111011

1001110 1110111

Ques

111000 111010111110110 111011

1001 110 11 111011 111

9 6 3 59 7 → gap list.

posting List : 9 15 18 77 84

{ gamma code of 1 is 0 }

Two gap sequences to be merged in blocked sort-based indexing.

gap sequence of run1 : 11101111111100111111111101000111101

gap sequence of run2 :

11111010000111111001000111110100001111010101

1110	1010111	11000111	101
14	87	199	5

Posting list : 14 101 300 305

110000	1001000	10100000	110101
48	72	160	53

Posting list : 48 120 280 333

RANKED RETRIEVAL VS BOOLEAN RETRIEVAL :

Parametric Index : meta data info ✓

doc : year 2002 author xyz

Zone Index : abstract , title , body

① william. abstract : 2, 5, 7, 20

william. title : 3, 7, 9, 23

william. body : 5, 7, 13, 22

② william → [2. abstract, 2. title] [5. body] [7. title] ---

Assume l number of zones.

$$g_1, g_2, \dots, g_l \in [0, 1]$$

$$\sum_{i=1}^l g_i = 1$$

S_i : boolean value term i^{th} zone.

* weighted zone score $= \sum_{i=1}^l g_i S_i$

Example

william :	2.abstract	2.title	5.body	10.abstract, 10.title 10.body
	0.2 abstract	0.3 title	0.5 body	

$$2 \rightarrow 0.2 \times 1 + 0.3 \times 1 = 0.5$$

$$5 \rightarrow 0.5 \times 1 = 0.5$$

$$10 \rightarrow 0.2 \times 1 + 0.3 \times 1 + 0.5 \times 1 = 1$$

* According to score we rank the term.

Document having max weight will be displayed first.

Ranked Retrieval

term weighting : free form

$$t_1, t_2, t_3 \quad d_1, d_2$$

① term frequency

$tf_{t,d}$ = no of time t occurred in d

ka term kisi document me kitni baar aya hai

② Inverse Document frequency (IDF) :

DF_t : no of documents having term t .

$$\boxed{idf_t = \log \frac{N}{df_t}}$$

$N \rightarrow$ no of documents in your collection.

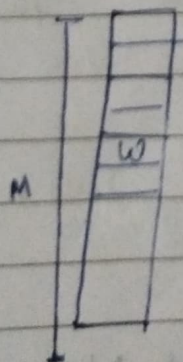
rare term $\rightarrow idf \uparrow$ (more weight)

frequent / common term $\rightarrow idf \downarrow$

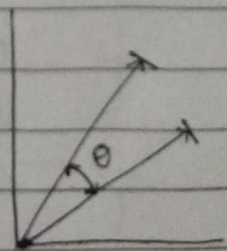
③ $tf \cdot idf$ = $tf_{t,d} \times \log \frac{N}{df_t}$

Vector Space Model :

compute similarity b/w two documents



tf $tf \cdot idf$



11 cosine similarity

$$\text{sim}(d_1, d_2) = \frac{\vec{v}(d_1) \cdot \vec{v}(d_2)}{|\vec{v}(d_1)| |\vec{v}(d_2)|}$$

$$\vec{x} \cdot \vec{y} = \sum_{i=1}^n x_i y_i$$

Ques

	tf	idf	idf	$W_{t,2}$	$\frac{tf \cdot idf - tf}{tf}$	W
auto	0	5000	2.30	0	1	1
car	1	5000	1.3	1.3	0	0
best	1	10000	2.0	2.0	1	1
insurance	1	1000	3.0	3.0	2	2

← query →

$N = 10,00,000$

query: 'best car insurance'

$$\text{idf} = \log \frac{N}{\text{dt.}}$$

$$\frac{1}{\sqrt{1^2 + 0^2 + 1^2 + 2^2}}$$

$W_{t,2}$	product
0.41	0
0	0
0.41	0.82
0.82	2.46
document	product

(query × document)

$$0 + 0 + 0.82 + 2.46$$

$$= 3.28$$

Similarity b/w query and document.

Variants of tf-idf functions:

1. Sublinear tf scaling (logarithmic tf)

$$w_{t,d} = \begin{cases} 1 + \log t_{f,t,d} & \text{if } t_{f,t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$$

2. Maximum tf normalization:
for a doc d

$$t_{f_{\max}}(d) = \max_{t \in d} t_{f,t,d} \quad \tau \text{ all term set.}$$

$$n t_{f,t,d} = a + [1-a] \frac{t_{f,t,d}}{t_{f_{\max}}(d)} \quad a = [0, 1]$$

3. digital cameras.

query

word	tf	wf	df	idf	$q_i = wf \times idf$	tf wt
digital	1	1	10000	3	3	
video	0	0	100000	2	0	
cameras	1	1	50000	2.30	2.3	

$$w_t = \begin{cases} 1 + \log t_f & \text{if } t_f > 0 \\ 0 & \text{otherwise} \end{cases}$$

Document: "Digital cameras and video cameras" and stop

word	tf	wt	df	$q_i = wf \times idf$ (Normalized wt).	word
digital	1	1	0.52		
video	1	1	0.52		
cameras	2	1.3	0.68		

$$\frac{wf}{\sqrt{1^2 + 1^2 + (1.3)^2}}$$

$$q_i \times d_i$$

$$1.56$$

$$1.560$$

$$1.56$$

$$\sum q_i d_i = 3.12$$

* Problem with cosine similarity based Ranked Retrieval
we have to compute similarity of every document
which is way too complex

* Inexact top k retrieval.

$$K < A \ll N$$

Content doc set

1. Index Elimination : idf threshold

→ (only compute those whose idf is greater than some (given) value).

2. Champion list:

→ pre computed set of document with highest weights.

$$t = \underbrace{\quad \quad \quad}_{\text{champion list}}$$

we use only top k for computation.
pre store only top k docs per term, query only those.

3. Static quality scores and ordering.

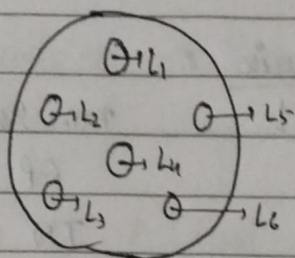
$g(d)$: (term ke kitna important)

$$\text{net score } (q, d) = g(d) + \vec{v}(d) \cdot \vec{v}(q)$$

4. Impact Ordering

if posting list is ordered by $g(d)/tf$
 so compute similarity for first x docs of posting list
 or when it drops below a threshold.

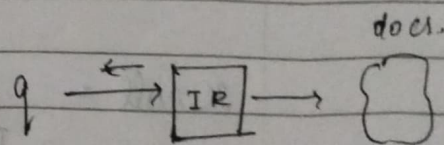
5. Cluster Pruning

 $\sqrt{n}$

- we have leader documents in every document
- we then compare leaders
- And at last we compute only of those who have more similarities of leader documents.

Evaluation of IR Systems:

- Fast
- Space
- Interface
- Relevant

Benchmark :

- ① Doc collection.
- ② Set of Info. need (query)
- ③ doc. Query relevance in binary for each pair of query and documents.

set based parameters for eval. of IR system.

① Precision : $\frac{\# \text{ relevant doc retrieved}}{\# \text{ doc. retrieved}}$

② Recall : $\frac{\# \text{ relevant doc. retrieved}}{\# \text{ relevant.}}$

Confusion	Matrix	
	relevant	non relevant
Retrieved	TP	FP
Non retrieved	FN	TN

T $\rightarrow$ True F $\rightarrow$ False
P $\rightarrow$ Positive N $\rightarrow$ Negative

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

F-Score Harmonic mean of P & R

$$F_1 = \frac{2PR}{P + R}$$

P $\rightarrow$ Precision
R $\rightarrow$ Recall.

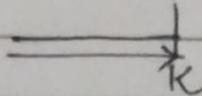
Ques

An IR system returns 8 relevant doc. and 10 non relevant docs. There are total 20 docs relevant in the collection. What is the precision of system in this search and what is recall?

$$\text{Precision} = \frac{8}{18} = 0.44$$

$$\text{Recall} = \frac{8}{20} = 0.4$$

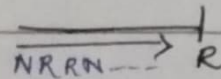
→ Precision at k

tells the precision at level k 

→ Precision at R

tells precision at R (total relevant docs)

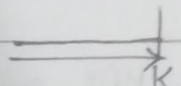
we fetch R docs.



$$\text{Precision} = \frac{8}{19} = 0.44$$

$$\text{Recall} = \frac{8}{20} = 0.4$$

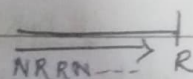
→ Precision at k

tells the precision at level k 

→ Precision at R

tells precision at R (total relevant docs)

we fetch R docs.



MAP (Mean Average Precision)

$$\text{MAP}(Q) = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{m_i} \sum_{k=1}^{m_i} \text{Precision}(R_{j_k})$$

$$\begin{array}{ccc} R & N & R & N & N & R \\ \downarrow & & \downarrow & & & \downarrow \\ \frac{1}{1} & & \frac{2}{3} & & & \frac{3}{6} \\ \hline & & 3 & & & \end{array} = \text{map.}$$

$$\frac{\frac{6}{6} + \frac{4}{6} + \frac{3}{6}}{3} = \frac{13}{18} = \text{MAP}(Q).$$

if multiple queries then divide by total number of queries.

	System 1				System 2			
Ques	R	N	R	NN	NN	R	R	
Highest Map?	↓	↓			↓	↓	↓	↓
	1	2	+		3	4	+	
	3				5	10		
	4				4			

0.60

0.49

MAP of system 1 is greater than MAP of system 2.

Q-Precision?	R	N	R	NN	NN	R	R	
	total relevant docs = 4				relevant docs = 4			
	2	1	=	0.5		1	=	0.25
	4	2				4		

It ranks the system the same as map

Ques	RR	NNN	NNN	RN	RNNNR	NNNNR
------	----	-----	-----	----	-------	-------

(i) Precision of system on top twenty = $\frac{6}{20} = \frac{3}{10}$

(ii) What F1 score = $\frac{2PR}{P+R} = \frac{2 \times \frac{3}{10} \times \frac{6}{8}}{\frac{3}{10} + \frac{6}{8}} = \frac{3}{7}$

$$\frac{2 \left(\frac{3}{10} \right)}{\frac{3}{10} + \frac{6}{8}} = \frac{9 \times 20}{20 \times 21} = \frac{3}{7}$$

c) what is uninterpolated precision of system at 25% recall?

25% recall : 1 Relevant.

RRNNN - - - - -

$$\frac{1}{2} = 0.5$$

d) map of query?

$$\Rightarrow \frac{1}{1} + \frac{2}{2} + \frac{3}{9} + \frac{4}{11} + \frac{5}{15} + \frac{6}{20} = 0.55$$

put - - -

e) Largest Possible MAP that this system could have

RRNNN - - - - - | RR

$$\frac{1}{2} + \frac{2}{9} + \frac{3}{11} + \frac{4}{15} + \frac{4}{20} + \frac{6}{21} + \frac{8}{22} = 8$$

starting on the right

least

L

RRNNN - - - - - | - - - - - RR

$$\frac{1}{2} + \frac{2}{9} + \frac{3}{11} + \frac{4}{15} + \frac{4}{20} + \frac{6}{21} + \frac{8}{22} = 8$$